

PWA & OFFLINE-FIRST ARCHITECTURE

Technical Interview Preparation Guide for React Engineers

1. WHAT IS REACT?

React is a JavaScript library used to build the frontend/user interface. It organizes the UI into reusable components, manages state reactively, and updates the DOM efficiently.

Normal React App Flow:

User Action → React UI (State Update) → HTTP/API Request → Backend Service → Database

2. WHAT IS A PWA?

Progressive Web App (PWA) — A web application that provides app-like features (installability, offline use, notifications) using modern browser capabilities.

Is PWA different from React?

Yes. **React** controls how the UI is rendered and structured. A **PWA** adds web capabilities to make it run like an app. React itself does *not* make an app a PWA.

React Application (UI / State)

+ PWA Features (Manifest + Service Worker + Cache + IndexedDB)

= React PWA (Offline-Capable Web App)

Core PWA Components:

- **Web App Manifest:** A JSON file specifying metadata (name, icons, start URL, theme colors) so the app can be installed.
- **Service Worker:** A client-side JavaScript file serving as a network proxy that manages resource caching and background tasks.
- **Caching:** Storing static application files (HTML, CSS, JS) locally to ensure fast, offline loads.
- **Offline Support:** Intercepting requests to serve cached assets and queuing data syncs when connection is offline.
- **Installability:** Enabling users to add the application directly to their mobile/desktop home screens without an app store.

3. HOW DOES A NORMAL WEB APP WORK?

A normal web application relies completely on real-time internet connectivity to function.

React UI → HTTP Request → Backend API → Database

If there is no internet: React UI × Backend API (The app breaks immediately, displaying a network connection error).

4. HOW DOES A PWA CHANGE THIS?

A PWA intercepts network requests and stores app resources locally, allowing the application to function without a network.

Normal App Architecture

UI → Internet → Server

Offline-Capable PWA Architecture

UI → Local resources / Local data (Service Worker / IndexedDB)

PWA does NOT make the backend or internet offline. It allows the frontend to continue working locally for functionality that has been designed for offline use by relying on cached files and client-side databases.

5. WHAT IS A SERVICE WORKER?

A **Service Worker** is a browser process that can work in the background and control certain network/cache behavior for a web app.

Browser / UI ↔ Service Worker ↔ Network / Cache

What it does: Caches application files, serves cached resources when appropriate, helps the app load/work offline, and handles background operations.

Does Service Worker store my health records? Not necessarily. The Service Worker mainly handles app/network/cache behavior, while **IndexedDB** stores application data.

6. WHAT IS INDEXEDDB?

IndexedDB is a database built into the browser for storing structured data locally.

AASHA Offline Record Flow:

ASHA enters health record → Internet unavailable → Store record in IndexedDB → Record stays locally

Feature	IndexedDB	localStorage
Capacity / Purpose	Stores structured/large application data (unlimited size constraints)	Mainly simple key-value data (5MB limit)
Suitability	Better for many records (structured search & retrieval)	Better for small simple values (themes, user settings)
AASHA Usage	Used for AASHA offline records (patient files, visit details)	Not our main record storage (only holds source device ID)

PWA & OFFLINE-FIRST ARCHITECTURE

Technical Interview Preparation Guide for React Engineers

7. WHAT DOES OFFLINE-FIRST MEAN?

Offline-first — The application is designed to work locally first and synchronize with the server later.

ONLINE	OFFLINE	INTERNET RETURNS
React → Backend → Database	React → IndexedDB	IndexedDB → Sync → Backend → Database

8. DOES THE BACKEND NEED TO BE DIFFERENT?

The backend does not need to become a 'PWA backend'. A PWA is primarily a frontend architecture. The backend still exposes normal REST APIs. The difference is that the frontend decides: 'Can I send this now?' If online, React → API → Backend; if offline, React → IndexedDB; later, IndexedDB → API → Backend.

Backend considerations (briefly):

- **Accept synchronized records:** Expose POST endpoints built to process batch payloads.
- **Handle duplicate requests safely:** Ignore or safely handle duplicates if the client retries an already saved record.
- **Validate and store:** Validate client-submitted structures before writing to database and return success/failure.

9. HOW DOES AASHA WORK OFFLINE?

AASHA Offline Flow: ASHA enters record → React → IndexedDB → Record waits locally → Internet returns → Sync mechanism → REST API → Spring Boot

db.js	Initializes IndexedDB ('asha-health-db') with stores for 'records', 'drafts', and 'metadata'. Handles raw database CRUD operations.
patientRepository.js	Coordinates NLP extraction, risk scoring, writing to IndexedDB, triggering background sync, and handles soft-deletes.
syncEngine.js	Verifies network state, filters unsynced records, and posts batches to `/api/sync` or sends DELETE requests. Marks records synced in DB.
HealthRecordController.java	Spring Boot REST controller exposing `@PostMapping("/sync")` for batch ingestion and `@DeleteMapping("/records/{id}")`.
vite.config.js	Configures the Workbox service worker options to register metadata and cache static CSS, JS, HTML, and web fonts.

10. HOW DOES SYNCHRONIZATION WORK?

Synchronization — Sending locally stored records to the backend when a connection is available.

Flow: 1. Find unsynced local records → 2. Send them to API → 3. Backend processes/stores them → 4. Successful records become synced

Failure Handling in AASHA: The frontend checks network connectivity using `verifyConnectivity()`. If sync fails or network is offline, the records remain in IndexedDB with status `pending`. No automated retry timer loops exist in `syncEngine.js`. Retry occurs dynamically on the next user action (saving a patient, opening the dashboard, or pulling records from the server).

11. WHAT HAPPENS IF THE INTERNET GOES OFFLINE?

Internet available: Record: React → API → Backend

Internet suddenly disappears: Record: React → IndexedDB

Internet returns: Record: IndexedDB → Sync → API → Backend (User does not lose records).

12. IMPORTANT TECHNICAL INTERVIEW QUESTIONS & QUICK ANSWERS

• **Q: Is a PWA the same as React?**

A: No. React builds the UI; PWA adds web capabilities such as installability, caching, and offline support.

• **Q: Does React automatically make an app a PWA?**

A: No. PWA features must be configured and implemented separately (manifest, service worker, databases).

• **Q: What is a Service Worker?**

A: A background browser process that helps control caching and network/background behavior.

• **Q: Where does offline application data go?**

A: In our case, structured offline records are stored locally in IndexedDB.

• **Q: Is IndexedDB the same as a Service Worker?**

A: No. IndexedDB stores structured data; a Service Worker handles network traffic and asset caching.

• **Q: Does PWA mean the backend works offline?**

A: No. The frontend runs offline using local files and data. The backend still requires network connectivity.

• **Q: Does the backend need to be different?**

A: Not fundamentally. It exposes standard REST APIs; the frontend handles queuing and synchronization logic.

• **Q: What happens when internet comes back?**

A: The frontend detects connection and synchronizes locally stored records with the backend.

• **Q: Why not use localStorage?**

A: IndexedDB is designed for structured, large collections. localStorage is blocking and limited to 5MB.

• **Q: What is offline-first?**

A: Designing the app so local operations work first, syncing with the server only when connectivity is established.

• **Q: Explain AASHA in one flow.**

A: React UI → IndexedDB (when offline) → Sync (when online) → REST API (Spring Boot) → PostgreSQL.